

JMDN Docker Execution Guide

Jupiter MetaZK Decentralized Network

1. Prerequisites

Before running the JMDN container, ensure the following are in place:

Requirement	Details
Docker	Docker Engine installed and running
jmdn.yaml	Your config file (copy from jmdn_default.yaml and edit)
Docker image	Built locally: <code>docker build -t jmdn:latest .</code>
Named volume	jmdn-data created automatically on first run

2. Build the Docker Image

Run this from the root of the jmdn repository:

```
docker build -t jmdn:latest .
```

To embed version metadata:

```
docker build \
  --build-arg GIT_COMMIT=$(git rev-parse --short HEAD) \
  --build-arg GIT_TAG=$(git describe --tags --always) \
  -t jmdn:latest .
```

3. Run — Minimal

Starts the node using the default config baked into the image. Suitable for quick testing only — peer identity is not persisted.

```
docker run -d --name jmdn \
  -p 6090:6090 \
  -p 6545:6545 \
  -p 6546:6546 \
  jmdn:latest
```

4. Run — Recommended

Mounts your own config and a named volume for persistent storage (ImmuDB data, peer identity, fastsync state).

■ Use \$(pwd)/jmdn.yaml — not a bare filename. Docker treats a bare name as a named volume (a directory), causing a mount error.

```
docker run -d --name jmdn \
  -p 6090:6090 \
  -p 6545:6545 \
  -p 6546:6546 \
  -p 16050:16050 \
  -p 16055:16055 \
  -p 16052:16052 \
  -v $(pwd)/jmdn.yaml:/etc/jmdn/jmdn.yaml:ro \
  -v jmdn-data:/opt/jmdn/data \
  jmdn:latest
```

5. Run — With Secrets via Environment Variables

Sensitive values should be passed as env vars rather than stored in jmdn.yaml. Thebe requires an external Postgres (port 5430) and Redis (6379).

```
docker run -d --name jmdn \
  -p 6090:6090 -p 6545:6545 -p 6546:6546 \
  -e JMDN_SECURITY_JWT_SECRET=your_jwt_secret \
  -e JMDN_SECURITY_EXPLORER_API_KEY=your_api_key \
  -e THEBE_SQL_DSN="postgres://jmdn:jmdn@host:5430/jmdn?sslmode=disable" \
  -e THEBE_REDIS_URL="redis://host:6379" \
  -v $(pwd)/jmdn.yaml:/etc/jmdn/jmdn.yaml:ro \
  -v jmdn-data:/opt/jmdn/data \
  jmdn:latest
```

6. Port Reference

Port	Service	Bind	Notes
6090	HTTP API / Explorer	0.0.0.0	Primary API endpoint
6545	Facade / JSON-RPC	0.0.0.0	Ethereum-compatible RPC
6546	WebSocket	0.0.0.0	WS subscriptions
16050	Block generation	0.0.0.0	Admin — blockgen
16055	Block propagation gRPC	0.0.0.0	P2P block sync
16052	DID service	0.0.0.0	Decentralized Identity
16053	CLI admin	127.0.0.1	Localhost only — not exposed
6050	Metrics	127.0.0.1	Localhost only — not exposed
6060	Profiler	127.0.0.1	Localhost only — not exposed
3322	ImmuDB	internal	Container-internal — not exposed

7. Container Startup Sequence

The entrypoint script (docker-entrypoint.sh) manages the startup order:

Step	Action
1	ImmuDB starts in the background (--dir /opt/jmdn/data)
2	Entrypoint polls port 3322 via nc until ImmuDB is ready (timeout: 30s)
3	start_jmdn_wrapper.sh is exec'd, which runs the jmdn binary
4	jmdn reads /etc/jmdn/jmdn.yaml and ./config/peer.json (from WORKDIR)

8. Useful Commands

View logs

```
docker logs -f jmdn
```

Shell into the container

```
docker exec -it jmdn bash
```

Stop and remove

```
docker stop jmdn && docker rm jmdn
```

Rebuild and restart

```
docker build -t jmdn:latest . && \  
docker rm -f jmdn && \  
docker run -d --name jmdn \  
-p 6090:6090 -p 6545:6545 -p 6546:6546 \  
-p 16050:16050 -p 16055:16055 -p 16052:16052 \  
-v $(pwd)/jmdn.yaml:/etc/jmdn/jmdn.yaml:ro \  
-v jmdn-data:/opt/jmdn/data \  
jmdn:latest
```

Inspect container config

```
docker inspect jmdn --format '{{.Config.Entrypoint}} {{.Config.Cmd}}'
```

9. Volume & Persistence

The named volume jmdn-data is mounted at /opt/jmdn/data inside the container. All persistent state lives here:

Path (inside container)	Contents
/opt/jmdn/data/data	ImmuDB database files
/opt/jmdn/data/config	peer.json — node identity (persists across restarts)
/opt/jmdn/data/DB	ImmuDB internal state

To back up the node identity: `docker cp jmdn:/opt/jmdn/data/config/peer.json ./peer.json.bak`

10. Health Check

The container has a built-in HEALTHCHECK that polls the API every 30 seconds:

```
curl -f http://localhost:6090/health
```

Check health status:

```
docker inspect --format='{{.State.Health.Status}}' jmdn
```